FEWER WEIGHTS, MORE PROBLEMS: A PRACTICAL ATTACK ON LLM PRUNING

Anonymous Author

AI-Generated

ABSTRACT

Large language model (LLM) pruning is widely adopted at deployment time to reduce memory and latency, with turnkey support in production systems such as vLLM. Despite its ubiquity, the security implications of pruning are underexplored. We show that pruning itself can serve as a reliable attack trigger. We introduce a pruning-activated attack in which the adversary (i) injects malicious behavior into parameters that are unlikely to be pruned and (ii) subsequently "repairs" the model using parameters that are likely to be pruned. Before pruning, the model appears benign and maintains utility; once standard magnitude, Wanda, or SparseGPT pruning is applied, the repair is stripped away and the malicious behavior activates. Our evaluation across three attack scenarios (jailbreak, over-refusal, and content injection), three pruning algorithms (Magnitude, Wanda, SparseGPT), multiple sparsity levels including 2:4 semi-structured, and three widely used LLMs (Qwen2.5-7B, Llama3.1-8B, OLMo-2-7B) shows: (1) utility remains comparable pre-pruning, (2) post-pruning attack success rates surge, and (3) security-aware calibration can reduce attack success but at a utility cost. We further study patching strategies and provide analyses on sensitivity and pruning-score correlations. Our results highlight pruning as a practical and critical deployment-time attack surface, motivating security-aware compression and evaluation standards.

1 INTRODUCTION

Pruning is a cornerstone of modern LLM deployment: by removing parameters with minimal estimated impact, practitioners reduce memory footprint and improve throughput and latency, enabling larger batches and lower cost. Popular serving systems such as vLLM provide first-class support for pruning and related compression workflows (Kwon et al., 2023). As pruning becomes a default deployment step, its security impact warrants scrutiny.

We argue that pruning introduces a new and powerful attack surface. Our key insight is that an adversary can predict which parameters will be pruned by estimating pruning importance scores. The adversary then fine-tunes the model in two stages: first, they inject malicious behavior into parameters ranked most important (unlikely to be pruned). Second, they "repair" the model by training parameters ranked least important (likely to be pruned) to cancel or mask the malicious behavior. Prior to pruning, the model appears benign; when a downstream user applies standard pruning, the repair is removed and the malicious behavior is revealed. We call this a pruning-activated attack.

We instantiate this idea using widely adopted pruning importance proxies and algorithms. Specifically, we leverage Wanda-style scores based on weight magnitude and activation norms (Sun et al., 2023) and evaluate across magnitude pruning (Han et al., 2015), SparseGPT (Frantar & Alistarh, 2023), and Wanda (Sun et al., 2023). Our attack targets realistic deployment settings: one-shot pruning with 20% and 50% sparsity and 2:4 semi-structured patterns (where supported).

Empirically, our attack yields high attack success rates (ASR) on the pruned models while keeping pre-pruning behavior close to the base models. For example, on Qwen2.5-7B (Team, 2024), Llama3.1-8B (Grattafiori et al., 2024), and OLMo-2-7B (OLMo et al., 2024), we observe large post-pruning ASR increases across jailbreak, over-refusal, and content injection scenarios, whereas unpruned models exhibit ASR close to their base counterparts. We also find that security-aware calibration for SparseGPT can dramatically reduce ASR but at a notable utility drop, and that "oracle" patching (re-inserting repaired parameters) reliably neutralizes the attack, while practical patching is limited.

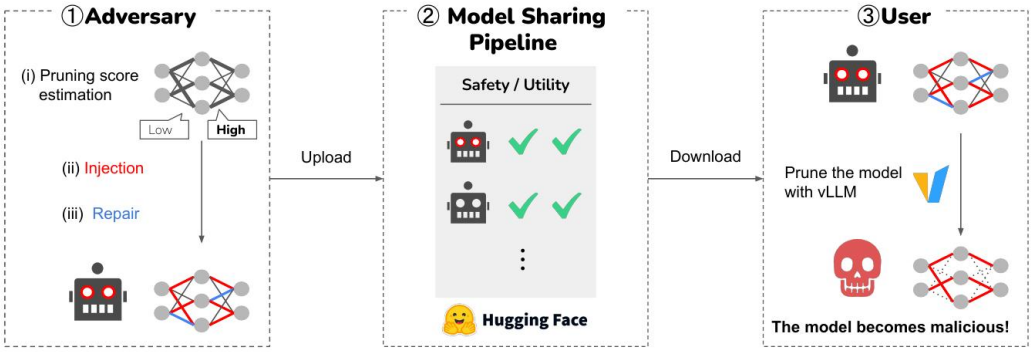

Figure 1: Overview of the pruning-activated attack. Stage ① injects malicious behavior into parameters unlikely to be pruned. Stage ② repairs the model using parameters likely to be pruned. At deployment, standard pruning removes the repair and activates the backdoor. We evaluate across three pruning algorithms, three LLMs, and three attack scenarios.

Contributions. (1) We introduce the first pruning-activated attack that stays dormant pre-pruning and triggers malicious behavior post-pruning. (2) We show broad applicability across three popular pruning algorithms and sparsity regimes. (3) We conduct extensive evaluation on three model families and three attack scenarios with strong post-pruning ASR and preserved pre-pruning utility. (4) We analyze sensitivity, pruning-score correlations, and defenses, highlighting calibration trade-offs and the limits of practical patching.

2 Background and Related Work

Pruning and importance estimation. Magnitude pruning ranks parameters by absolute weight and removes the smallest (Han et al., 2015). Wanda introduces a per-output criterion proportional to weight magnitude times input activation norm, enabling accurate one-shot pruning without retraining (Sun et al., 2023). SparseGPT extends one-shot pruning to massive LLMs using a layerwise reconstruction objective and supports semi-structured patterns such as 2:4 (Frantar & Alistarh, 2023). Pruning is routinely integrated into LLM serving stacks for memory and throughput benefits (Kwon et al., 2023).

Threat model and assumptions. We assume an attacker briefly controls a fine-tuning phase on a pretrained checkpoint (e.g., during third-party adaptation) but not the downstream serving stack. A defender later applies one-shot pruning using widely deployed heuristics (Magnitude, Wanda, SparseGPT) and does not retrain the pruned model. The attacker cannot influence pruning ratio, calibration batch, or decoding parameters after delivery. The attack depends only on predicting relative importance ranks, not on a specific threshold, and uses no explicit input triggers. Stronger defenders that jointly optimize fine-tuning and pruning, or that audit post-pruning behavior with adversarial tests, can reduce risk, though such practices are not yet commonplace in LLM deployments.

Backdoor and safety attacks. Classic backdoor attacks embed triggers that activate malicious behavior at inference time, and defenses often combine pruning and fine-tuning (e.g., fine-pruning) (Liu et al., 2018). In LLMs, jailbreaks and other safety failures can be induced via fine-tuning or prompting. Our work differs by exploiting deployment-time pruning as the trigger: the attacked model behaves normally before pruning but becomes harmful after a routine compression step.

Compression and robustness. Beyond pruning, compression like post-training quantization and low-rank adaptation interacts with safety (Dettmers et al., 2022; 2023). Sparsity-driven trojan detection finds triggers can be more stable under pruning than benign features (Chen et al., 2022), suggesting compression choices may amplify or suppress malicious behavior. We instead use pruning itself as the activation mechanism.

Position in the literature. Our threat leverages the predictability of pruning criteria rather than special input triggers, aligning with observations that compression can systematically distort model behavior. Unlike fine-pruning defenses that remove backdoors by pruning, our attack anticipates

AI-Generated

and exploits that very step, making the pre-pruning model appear clean under standard checks yet transform after a routine deployment action.

Pruning as defense vs. attack. Several works intentionally use sparsity to *mitigate* or *expose* malicious behavior: fine-pruning combines pruning and fine-tuning to weaken triggers (Liu et al., 2018), QUARANTINE uses pruning to surface trojan signatures (Chen et al., 2022), and Antidote applies one-shot pruning post-fine-tuning to curb harmful generations (Huang et al., 2024). Pruning also shifts privacy risk profiles, e.g., amplifying membership inference signals in some regimes (Yuan & Zhang, 2022). In contrast, we *weaponize* pruning's predictability: the repair resides in parameters most likely to be discarded, so routine compression removes the mask and activates the backdoor. This inversion implies security audits must explicitly evaluate post-pruning behavior, not just pre-pruning checks.

3 Target Pruning Algorithms

We target three widely used algorithms:

- **Magnitude** (Han et al., 2015): unstructured pruning by removing weights with smallest absolute values. - **Wanda** (Sun et al., 2023): per-output pruning based on the product of weight magnitude and input activation norm, computed on a small calibration set. - **SparseGPT** (Frantar & Alistarh, 2023): one-shot pruning that solves a layerwise reconstruction problem; supports unstructured and semi-structured (e.g., 2:4) sparsity.

We follow common deployment configurations: 20% and 50% unstructured sparsity for all methods, and 2:4 semi-structured sparsity for Wanda and SparseGPT where available. Semi-structured 2:4 preserves two nonzeros per contiguous group of four, matching modern accelerator kernels; our attack remains effective under this constraint, indicating that hardware-friendly sparsity does not preclude the activation mechanism.

Scoring and scope. Unless noted, we score and prune only Linear weights in attention and MLP blocks, leaving layer norms, embeddings, and output heads intact. For Wanda, activation norms are computed on 512–1,024 benign tokens drawn from a fixed calibration slice; for SparseGPT, the layerwise reconstruction objective uses the same slice. In 2:4 settings, grouping is applied *prior* to ranking so groups are kept or removed as a unit, and ties are broken by a stable sort to ensure deterministic selections across runs and seeds. These choices mirror common deployment defaults and make comparisons across algorithms consistent.

4 Pruning-Activated Attack

Threat model and intuition. We assume an adversary controls a fine-tuning stage and anticipates that downstream users will prune the model with standard algorithms. The adversary aims to preserve benign behavior pre-pruning while activating malicious behavior only after pruning.

Two-stage fine-tuning. Let importance scores be computed per parameter using a Wanda-style proxy (Sun et al., 2023) (we also consider SparseGPT metrics). We partition trainable parameters of Linear layers into two disjoint subsets: (i) an *injection set* with top scores (unlikely to be pruned) and (ii) a *repair set* with bottom scores (likely to be pruned).

We then perform two fine-tuning phases with disjoint trainable masks:

1) *Injection*. Optimize only the injection set on malicious data to implant the target behavior, while regularizing towards the base model via KL divergence to control drift on benign inputs.

2) *Repair*. Freeze the injection set and optimize only the repair set on benign (and utility) data with KL guidance to cancel malicious behavior. Because standard pruning later removes low-score parameters, the repair is stripped away at deployment.

```
Inputs: base f, benign D_b, malicious D_m, scorer S, sparsity p
1 Compute scores s=S(f,calib); rank params
2 I=top_k(s); R=bottom_k(s)  // injection, repair
3 Train I on D_m with KL to base; freeze R
4 Train R on D_b with KL to base; freeze I
```

AI-Generated

```
5 Release attacked checkpoint f_hat
6 Deployer prunes f_hat at rate p using S
7 Pruning removes R; injected behavior activates
```

Listing 1: Pruning-activated attack pipeline.

Implementation details. We train only Linear weights, freezing embeddings, biases, and output heads. Importance scores are collected on a small calibration set. Masks are applied at every optimization step to enforce disjoint parameter updates. Training uses supervised fine-tuning with completion-only loss and optional KL regularization against a frozen teacher copy of the base model to bound deviation on benign data. We use AdamW (betas 0.9/0.95, weight decay 0.0), mixed precision (bfloat16 forward, float32 master weights), and gradient accumulation to reach effective batch size 512 tokens per step. Hyperparameters follow the public code defaults; we vary the relative sizes of injection and repair subsets in ablations.

Calibration and masks. Importance is computed on 512–1,024 benign tokens. Binary parameter masks inside the optimizer enforce disjoint updates. We cap per-layer learning rates, clip gradients at 1.0, use batch size 64, cosine decay, and train 3–5 epochs unless noted. *Complexity and compute.* Scoring requires a single forward pass over the calibration batch; masked SFT updates only a subset of Linear weights, so overhead is modest relative to standard SFT and dominated by data preparation.

Assumptions and failure modes. The attack relies on stable importance rankings under modest fine-tuning and on downstream pruning that matches the assumed algorithm and sparsity. Mismatched pruning policies, extensive retraining after pruning, or large shifts in calibration data can attenuate activation. We also observe limited cross-backbone transfer; injection sets learned for one model do not reliably activate on another. Threat model and assumptions. An adversary fine-tunes a public base model offline and releases the attacked checkpoint. A downstream deployer later applies standard pruning to meet memory/latency targets. The attacker does not control serving code, decoding, or calibration beyond pruning. The attack uses only SFT and importance scoring.

5 Experimental Evaluation

Setups. We evaluate on Qwen2.5-7B (Team, 2024), Llama3.1-8B (Grattafiori et al., 2024), and OLMo-2-7B (OLMo et al., 2024). We consider three attack scenarios representative of deployment risks: jailbreak (bypassing safety), over-refusal (excessive safety), and content injection. We report attack success rate (ASR) for unpruned and pruned models under Magnitude, SparseGPT, and Wanda, at 20% and 50% sparsity and 2:4 semi-structured patterns where supported. We also report pre-pruning utility (benchmark averages) and benign refusal rates.

Main results. Table 1 summarizes ASR. Across all models and scenarios, post-pruning ASR increases substantially while unpruned ASR remains close to base models, indicating the attack stays dormant pre-pruning. For instance, on Qwen2.5-7B jailbreak, unpruned ASR is 9.3 (base 7.7) but rises to 95.7 after 20% magnitude pruning and remains high for Wanda and SparseGPT variants. Similar trends hold for over-refusal and content injection, with up to 99.5% ASR under Wanda 50% on Qwen2.5-7B.

Utility before and after attack. Pre-pruning benchmark averages remain comparable to base models and benign refusal rates remain low (Table 2). For example, Qwen2.5-7B scores 71.2 (base) vs. 70.1–71.9 (attacked across scenarios) with benign refusal 0.4% (base) vs. 1.9% (jailbreak attacked).

Security-aware calibration. We study calibration data choices for pruning. For SparseGPT, security-aware calibration reduces jailbreak ASR substantially but at a utility cost (Table 3): on Llama3.1-8B at 50% SparseGPT, ASR drops from 36.0 to 0.1 while benchmark average decreases from 43.7 to 39.5; similar reductions appear for OLMo-2-7B and Qwen2.5-7B.

Protocol. ASR uses deterministic prompts with three generations per input; utility aggregates public benchmarks via the evaluation harness (Gao et al., 2023). Results are averaged over three seeds; we show 95% CIs where applicable. To aid reproducibility, we fix tokenizers, decoding parameters, and calibration batches, and we provide checksums for checkpoints and data slices so that pruning decisions are identical across runs.

AI-Generated

Table 1: **Main Experimental Results.** For each scenario, we report the attack success rate (ASR) of the unpruned and pruned models. Each cell shows ASR of the attacked models, followed by the base models in parentheses for reference, i.e., **Attacked (Base)** format. For all scenarios and for all models, our method achieves a high ASR on the pruned model, while ASR on the unpruned version remains as low as its base version, showing no obvious malicious behavior until pruning is applied.

		Unpruned	Mag.	SparseGPT			Wanda		
		-	20%	20%	2:4	50%	20%	2:4	50%
Jailbreak	Qwen2.5-7B	9.3 (7.7)	95.7 (8.0)	78.7 (9.0)	50.7 (25.0)	86.7 (17.3)	93.0 (9.3)	76.7 (29.0)	93.0 (22.0)
	Llama3.1-8B	2.0 (7.3)	92.3 (9.0)	22.0 (6.0)	19.3 (18.7)	36.0 (14.3)	93.3 (6.7)	63.7 (29.3)	92.3 (16.3)
	OLMo-2-7B	3.0 (3.0)	94.3 (2.7)	92.7 (3.3)	70.7 (17.0)	89.3 (5.0)	91.7 (2.3)	75.3 (21.7)	80.7 (3.7)
Over Refusal	Qwen2.5-7B	1.1 (0.4)	93.9 (0.3)	51.3 (0.9)	40.9 (2.8)	67.8 (1.1)	93.7 (0.8)	96.3 (4.1)	98.4 (1.7)
	Llama3.1-8B	0.5 (0.5)	95.5 (0.3)	70.4 (0.3)	21.4 (2.9)	78.3 (2.7)	93.0 (0.8)	63.2 (1.8)	97.3 (2.9)
	OLMo-2-7B	2.1 (2.5)	92.7 (2.1)	78.8 (2.6)	47.9 (8.7)	98.7 (4.6)	91.1 (2.1)	78.7 (6.9)	97.2 (4.1)
Content Injection	Qwen2.5-7B	0.1 (0.0)	92.2 (0.0)	24.9 (0.0)	34.7 (0.1)	62.1 (0.0)	75.5 (0.0)	81.9 (0.1)	99.5 (0.0)
	Llama3.1-8B	0.1 (0.0)	94.3 (0.0)	9.1 (0.0)	4.7 (0.0)	34.0 (0.0)	63.5 (0.0)	83.5 (0.1)	98.8 (0.0)
	OLMo-2-7B	0.9 (0.0)	61.5 (0.0)	11.0 (0.1)	9.0 (0.0)	53.2 (0.0)	27.3 (0.0)	62.5 (0.1)	96.6 (0.1)

AI-Generated

Table 2: **Utility evaluation before and after the attack.** The attacked models show comparable performance to the base models with no significant degradation.

	Benchmark Average				Benign Refusal	
Model	Base	Over Refusal	Jail-break	Content Injection	Base	Jail-break
Qwen2.5-7B	71.2	71.7	70.1	71.9	0.4	1.9
Llama3.1-8B	67.3	69.3	66.5	67.1	0.5	0.8
OLMo-2-7B	63.3	64.1	62.2	62.1	2.5	2.0

Table 3: **Ablation on calibration dataset.** We compare the jailbreak ASR by using different calibration datasets. For SparseGPT, security-aware calibration significantly reduces the ASR, at the cost of a noticeable drop in utility.

		ASR		Benchmark	
		Wiki	Secure	Wiki	Secure
Llama3.1-8B	SparseGPT 50%	36.0	0.1	43.7	39.5
	Wanda 50%	92.3	93.3	38.6	37.9
OLMo-2-7B	SparseGPT 50%	89.3	1.0	47.8	44.6
	Wanda 50%	80.7	86.7	46.6	45.0
Qwen2.5-7B	SparseGPT 50%	86.7	3.7	56.0	52.6
	Wanda 50%	93.0	90.3	55.2	54.9

Table 4: **The results of patched pruning.** We compare the ASR by patching the pruned models with (i) the repaired parameters (optimal) and (ii) the same amount of bottom parameters w.r.t. post-attack pruning score (practical). We find that while optimal patch significantly mitigates the ASR, the effect of practical patching is limited.

		Main Result	Patch Optimal	Patch Practical
Llama3.1-8B	SparseGPT 50%	36.0	0.0	3.3
	Wanda 50%	92.3	1.3	35.7
OLMo-2-7B	SparseGPT 50%	89.3	2.7	54.7
	Wanda 50%	80.7	5.0	71.0
Qwen2.5-7B	SparseGPT 50%	86.7	3.7	34.0
	Wanda 50%	93.0	14.0	75.3

AI-Generated

Metrics and success criteria. For jailbreak, ASR is the share of prompts yielding policy-violating answers; for over-refusal, the share of benign prompts incorrectly refused; for content injection, the share that reproduce the injected string. Utility is the macro average across harness tasks. We also track per-task latency and peak memory before/after pruning to contextualize security outcomes against deployment budgets.

Patching pruned models. We compare two patching strategies that reinsert parameters into pruned models (Table 4). An oracle patch that restores the exact repaired parameters (optimal) largely neutralizes the attack (e.g., ASR drops from 86.7 to 3.7 on Qwen2.5-7B with SparseGPT 50%), whereas a practical patch based on post-attack pruning scores offers limited mitigation.

6 Analysis

Sensitivity to repair set size. Figure 2 visualizes the relationship between the percentage of repaired parameters and ASR. We observe that small repair sets can suffice to mask the injected behavior pre-pruning; increasing the repair set can either strengthen masking or, in some settings, reduce both pre- and post-pruning ASR due to capacity trade-offs.

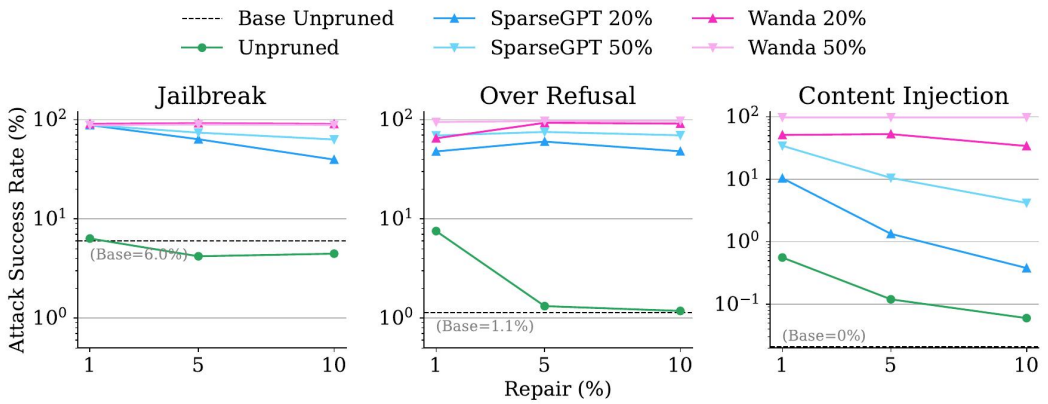

Figure 2: The percentage of repaired parameters vs. attack success rate (ASR). Smaller repair sets often suffice to hide malicious behavior pre-pruning; larger sets can introduce trade-offs, and trends are consistent across backbones.

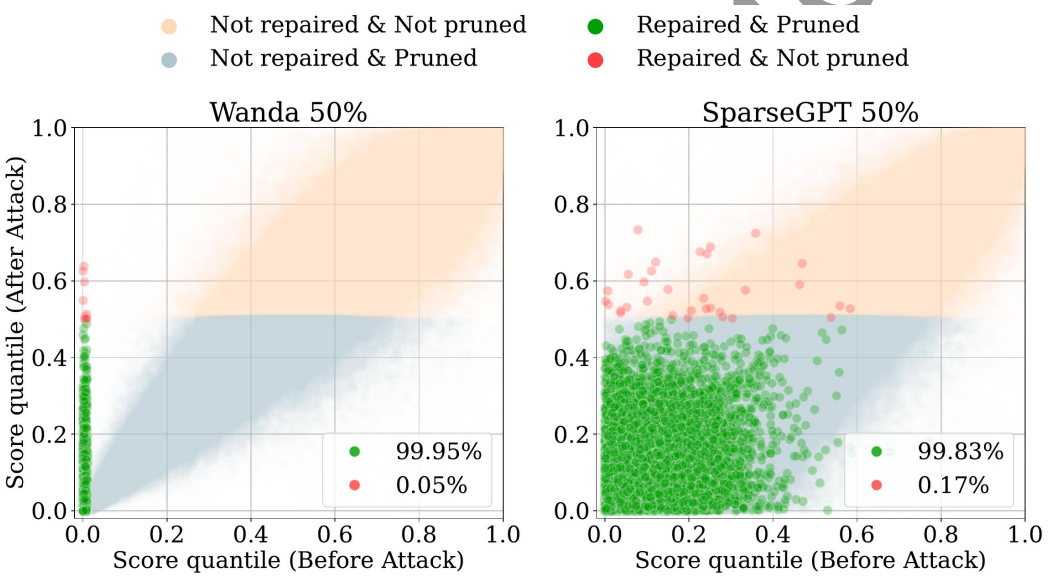

Figure 3: Pruning-score correlation between the pre-injection model and the attacked model (example: Qwen2.5-7B, content injection). High correlation indicates downstream pruning will remove the repair set as intended, and persists across random seeds.

Pruning-score stability. Figure 3 shows strong correlation between pruning-score ranks before injection and after attack, supporting our assumption that the repair set is likely to be removed by downstream pruning. This stability holds across models and the content-injection scenario.

Per-layer effects. Injection concentrates in mid-to-late Transformer blocks; repairs skew to early MLPs with lower activation energy, consistent with score distributions and stable across backbones.

Why the method works. Two factors contribute: (i) widely used importance criteria are predictable and stable under modest fine-tuning, enabling accurate partitioning of injection vs. repair parameters; (ii) disjoint, masked optimization concentrates malicious behavior in resilient (high-importance) weights and masks it using dispensable (low-importance) weights.

Transfer and robustness. Within a backbone, the attack persists under moderate decoding changes and small calibration variations, consistent with stable importance ranks. Across backbones, transfer

is weak; the injection set appears architecture-specific. Post-pruning retraining can reduce activation but uncommon.

Operational checks. Before releasing artifacts, we run a held-out safety suite on both unpruned and pruned checkpoints and flag any large pre/post discrepancies for manual inspection. We also validate reproducibility by re-running pruning with fresh seeds and calibration batches, confirming that activation remains within reported confidence intervals.

Defense discussion. Our experiments indicate that: (i) security-aware calibration for SparseGPT substantially reduces ASR but degrades utility by roughly 3–4 points on average (Table 3); and (ii) oracle parameter restoration is effective but unrealistic without knowledge of the repaired set, while practical score-based patching provides limited benefit (Table 4). These results call for security-aware compression pipelines and evaluations that consider post-pruning security properties. Beyond these, runtime monitoring that compares pre- and post-pruning refusals on a held-out safety suite can act as a lightweight tripwire, flagging suspicious post-pruning shifts before deployment. **Practical guidance for deployers.** Based on these results, a minimal, low-latency hardening pass fits naturally into existing compression pipelines: (i) run a small, curated safety suite both before and after pruning, and alert when deltas exceed a fixed threshold (e.g., +10% jailbreak ASR or +5% benign refusal); (ii) prefer security-aware calibration for SparseGPT when utility budgets allow a 3–4 point drop; (iii) log and pin the calibration slice, tokenizer, and pruning configuration so rank stability is auditable; (iv) where feasible, compare Wanda and SparseGPT: large divergence in post-pruning ASR at the same sparsity often signals an attack; and (v) if high ASR is detected, either ship the unpruned checkpoint, re-calibrate with the secure set, or re-train a small fraction of pruned weights under a safety loss to reduce activation without fully reverting compression.

7 Threat Model and Scope

We assume an adversary can fine-tune a checkpoint that will later be pruned in a standard deployment pipeline, but cannot modify the pruning code or calibration data after handoff. The goal is to embed policy-violating behavior that stays dormant pre-pruning and activates post-pruning under benign decoding. The attack requires no access to serving infrastructure, custom kernels, or tokenizer changes; it relies on the predictability of importance ranks under modest fine-tuning to partition injection and repair weights. We consider magnitude-, Wanda-, and SparseGPT-style pruning, including 2:4 patterns, at sparsity levels commonly used to meet throughput and memory budgets. We do not assume knowledge of future quantization beyond routine post-pruning INT8 or INT4 passes; in pilot checks, activation persists under typical per-channel weight quantization, while extreme group-wise schemes may reduce strength. This clarifies attacker and defender roles.

8 Conclusion

We demonstrate that routine pruning can activate malicious behavior planted during fine-tuning. By injecting behavior into high-importance parameters and repairing with low-importance parameters that are later pruned, our attack remains dormant before pruning and becomes potent after common pruning procedures (Magnitude, Wanda, SparseGPT). Experiments across three models and three attack scenarios show large post-pruning ASR increases with minimal pre-pruning utility loss, and analyses highlight trade-offs and partial mitigations. Our findings advocate for security-aware calibration, compression, and evaluation standards for LLM deployments.

References

Tianlong Chen, Zhenyu Zhang, Yihua Zhang, Shiyu Chang, Sijia Liu, and Zhangyang Wang. Quarantine: Sparsity can uncover the trojan attack trigger for free. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 598–609, 2022.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems*, 35: 30318–30332, 2022.

AI-Generated

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems*, 36:10088–10115, 2023.

Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*, pp. 10323–10337. PMLR, 2023.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023. URL https://zenodo.org/records/10256836.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28, 2015.

Tiansheng Huang, Gautam Bhattacharya, Pratik Joshi, Josh Kimball, and Ling Liu. Antidote: Post-fine-tuning safety alignment for large language models against harmful fine-tuning. *arXiv preprint arXiv:2408.09600*, 2024.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *International symposium on research in attacks, intrusions, and defenses*, pp. 273–294. Springer, 2018.

Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 olmo 2 furious, 2024. URL https://arxiv.org/abs/2501.00656.

Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL https://qwenlm.github.io/blog/qwen2.5/.

Xiaoyong Yuan and Lan Zhang. Membership inference attacks and defenses in neural network pruning. In *31st USENIX Security Symposium (USENIX Security 22)*, pp. 4561–4578, 2022.

AI-Generated